Software Requirements Specification (SRS)

Railway Reservation System

Project: Railway Reservation System (Booking & Cancellation)
Version: 1.0
Author: Pranav B N, Samaresh, Amruta Karoshi, Manapriya Kudurebyalya — Team 15
Date: 13-09-2026
Status: Draft for Review

Revision History

Version	Date	Author	Change Summary	Approval
0.1	05-09-2026	Team 15	Initial draft — scope, actors and rough feature list sketched out after finalising the topic.	–
1.0	12-09-2026	Team 15	Full SRS with functional, non-functional and security requirements, RTM and use-case diagrams.	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features (Detailed)
5. Non-Functional Requirements
5.1 Security
6. Quality Attributes & Acceptance Tests
7. System Models and Diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document lays out the Software Requirements Specification for a Railway Reservation System built in C/C++. The project was picked up as a systems-programming exercise — the goal is to model the core workflow of a real reservation counter (searching trains, booking a seat, cancelling it and getting a refund) using file handling and data structures instead of a full database back-end. The SRS is written for the course instructor/evaluator as well as for the author's own reference while implementing the modules, so that the code and the write-up stay consistent with each other.

1.2 Scope

The system covers passenger-facing operations — searching for trains between two stations, checking seat availability across classes, booking a ticket, generating a PNR, cancelling a ticket and receiving a refund estimate — along with a small admin/clerk console for adding trains, updating fares and quotas, and viewing booking reports. Persistence is handled through structured binary/text files (no external DBMS), which keeps the project within the scope of a C/C++ file-handling assignment. Payment is simulated (no real gateway integration); the system only mimics the transaction step and generates a receipt. Multi-user concurrent access, mobile apps, and integration with the actual Indian Railways (IRCTC) infrastructure are explicitly out of scope.

1.3 Audience

Course instructor/evaluator, the student developer (also the sole maintainer), and any peer reviewer asked to test the build before submission.

1.4 Definitions, Acronyms and Abbreviations

- PNR — Passenger Name Record, the unique reference generated for every booking.
- RAC — Reservation Against Cancellation, a partially-confirmed seat status.
- WL — Waitlist, a queued booking with no seat yet assigned.
- CLI — Command Line Interface, the primary user interface for this project.
- SRS — Software Requirements Specification, this document.
- FR / NFR — Functional Requirement / Non-Functional Requirement.

2. Overall Description

2.1 Product Perspective

The Railway Reservation System is a standalone, self-contained application — there is no existing product it replaces or plugs into. It is structured as a console program with a layered internal design: an I/O/menu layer, a business-logic layer (availability checks, fare calculation, PNR generation, waitlist promotion) and a storage layer that reads and writes fixed-format records to disk files (trains.dat, bookings.dat, users.dat). This mirrors, at a much smaller scale, how a real reservation system separates the counter UI from the booking engine and the database.

2.2 Major Product Functions

- Passenger registration and login
- Train search by source, destination and date
- Seat/berth availability check across classes (Sleeper, AC 3-Tier, AC 2-Tier, General)
- Ticket booking with automatic PNR generation
- Simulated payment and receipt generation
- Ticket cancellation with refund calculation based on time-to-departure
- Waitlist and RAC queuing with automatic promotion when a confirmed seat is freed
- PNR status lookup
- Admin functions: add/update/remove trains, set fares and quotas, view booking and revenue reports
- Data persistence via file handling, with a basic backup/restore utility

2.3 User Roles and Characteristics

- Passenger — the primary user; assumed to have basic computer literacy and no special training. Wants a quick, unambiguous booking flow.
- Reservation Clerk / Admin — manages train data, fares and quotas, and reviews reports; is assumed to be comfortable with a menu-driven console tool.

2.4 Operating Environment

A single-user desktop/laptop running Windows or Linux, compiled with GCC/G++ (or an equivalent C/C++17-capable compiler). No network connectivity is required for the core flow since the "payment gateway" and "bank" are simulated within the program itself. Data files live on the local filesystem alongside the executable.

2.5 Constraints

- No external database — all persistence must go through C/C++ file I/O (fopen/fstream, fixed-size structs).
- Single-user, single-process access; no locking mechanism for concurrent writers.
- Console-based UI only — no graphical front-end in this version.
- Payment is simulated; no PCI-DSS or real financial integration applies, but card-like input (a dummy 16-digit number) should still be masked on screen for realism.
- Project timeline and academic scope limit the system to a single train class table per train rather than a fully normalised fare/route database.

3. External Interface Requirements

3.1 User Interfaces

A numbered, menu-driven CLI is the only interface. Every screen shows the current menu, a short prompt, and validates input before proceeding (e.g. rejecting a non-numeric seat count). Error messages state what went wrong and what the user can try next, rather than raw error codes. Where the console supports it, tabular output (train list, seat map, booking summary) is column-aligned for readability.

3.2 Hardware Interfaces

- Standard keyboard input and monitor/terminal output — no specialised hardware is used.
- Local disk storage for data files (trains.dat, bookings.dat, users.dat, admin_log.txt).

3.3 Software Interfaces

- C standard library / C++ STL for data structures (arrays, linked lists or vectors for waitlist queues).
- File I/O APIs (fstream / stdio.h) for all persistence — acts as the stand-in for a database layer.
- A simulated Payment Module (an internal function, not an external API) that mimics gateway request/response to keep the payment step realistic without needing network access.

3.4 Communications Interfaces

None required for normal operation — the application is fully offline. If a future version adds networked multi-terminal support, it would use TCP sockets between a client console and a central booking server; this is noted as a possible extension but is not part of the current scope.

4. System Features (Detailed)

Each feature below lists its functional requirements with an acceptance criterion and a reference test case. IDs follow the pattern RRS-F-###. Across all features, this SRS defines 23 functional requirements, comfortably above the minimum of 15 asked for.

4.1 Authentication

Description: Lets a passenger create a local profile and log in before booking; lets the admin log in with elevated privileges.

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-001	The system shall allow a new passenger to register with name, age, gender, contact number and a password, storing the record in users.dat.	Functional	High	Passenger	AC-001: A new user record is retrievable immediately after registration. Test: TC-Reg-01	Password should be stored hashed, not plain text
RRS-F-002	The system shall authenticate a passenger by username and password before allowing booking or cancellation.	Functional	High	Passenger	AC-002: Correct credentials log the user in; incorrect ones are rejected with a clear message. Test: TC-Auth-01	Depends on RRS-F-001
RRS-F-003	The system shall provide a separate admin login using a pre-configured credential set, distinct from passenger accounts.	Functional	High	Admin	AC-003: Admin menu is inaccessible without successful admin login. Test: TC-Auth-02	Credentials read from a protected config, not hard-coded in cleartext where avoidable

4.2 Train Search & Seat Availability

Description: Lets a passenger look up trains between two stations and see live seat counts per class before committing to a booking.

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-010	The system shall let a passenger search trains by entering source station, destination station and travel date.	Functional	High	Passenger	AC-010: Matching trains are listed with train number, name and timings. Test: TC-Search-01	Requires trains.dat populated by admin
RRS-F-011	The system shall display seat availability per class (Sleeper, AC 3-Tier, AC 2-Tier, General) for each matching train.	Functional	High	Passenger	AC-011: Displayed counts match the underlying record at query time. Test: TC-Search-02	Depends on RRS-F-010
RRS-F-012	The system shall indicate when a class is full and offer RAC/Waitlist as the booking option instead of blocking the request outright.	Functional	Medium	Passenger	AC-012: A full class shows “WL available” rather than simply refusing the search. Test: TC-Search-03	Feeds into RRS-F-023 (waitlist)

4.3 Ticket Booking

Description: Captures passenger and journey details, confirms or queues the seat, and generates a PNR and receipt.

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-020	The system shall collect passenger name, age, gender and berth preference for each booking.	Functional	High	Passenger	AC-020: All fields are validated (non-empty, age is numeric and in range 1–120). Test: TC-Book-01	–
RRS-F-021	The system shall generate a unique 10-digit PNR for every successful booking	Functional	High	Passenger / Business	AC-021: No two bookings ever share a PNR across a full test run. Test: TC-Book-02	PNR generation logic must avoid collisions after file reload

	and persist it with the booking record.					
RRS-F-022	The system shall decrement the available seat count for the chosen class immediately on confirmed booking.	Functional	High	Business	AC-022: Seat count in trains.dat reflects the new total after booking. Test: TC-Book-03	Must stay consistent with RRS-F-011
RRS-F-023	The system shall place a booking on RAC or Waitlist, with a queue position, when the requested class has no confirmed seat left.	Functional	High	Business	AC-023: A WL booking shows a position number; position decreases as earlier bookings are cancelled. Test: TC-Book-04	Depends on RRS-F-012, RRS-F-032
RRS-F-024	The system shall simulate a payment step (dummy card/UPI entry) and only confirm the booking after a successful simulated authorisation.	Functional	Medium	Passenger	AC-024: A simulated payment failure leaves no booking record behind. Test: TC-Book-05	Card-like input masked on screen
RRS-F-025	The system shall print/display a formatted ticket receipt containing PNR, passenger details, train, class, seat/berth number and fare.	Functional	High	Passenger	AC-025: Receipt fields match the stored booking record exactly. Test: TC-Book-06	–

4.4 Ticket Cancellation & Refund

Description: Lets a passenger cancel a confirmed or waitlisted ticket and receive a refund figure based on how close to departure the cancellation happens.

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-030	The system shall allow a passenger to cancel a ticket by entering the PNR, after verifying it belongs to the logged-in user.	Functional	High	Passenger	AC-030: Cancelling a PNR that isn't owned by the current user is rejected. Test: TC-Cancel-01	Depends on RRS-F-002, RRS-F-021
RRS-F-031	The system shall compute a refund percentage based on the time remaining before departure (e.g. >24h: 75% refund, 12–24h: 50%, <12h: 25%).	Functional	High	Business	AC-031: Refund shown matches the slab for the simulated system clock/departure time. Test: TC-Cancel-02	Refund slabs configurable in a constants file
RRS-F-032	The system shall free the cancelled seat and automatically promote the next Waitlist/RAC passenger, updating their status and notifying them of the change (on-screen, since the system is offline).	Functional	High	Business	AC-032: The first WL entry moves to RAC/Confirmed immediately after a matching cancellation. Test: TC-Cancel-03	Depends on RRS-F-023
RRS-F-033	The system shall mark a cancelled booking's record as CANCELLED rather than deleting it, preserving it for reporting.	Functional	Medium	Admin	AC-033: Cancelled bookings still appear in admin reports with status CANCELLED. Test: TC-Cancel-04	Feeds RRS-F-043

4.5 PNR Status & Ticket Lookup

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-034	The system shall let a passenger check the current status (Confirmed / RAC / WL-n / Cancelled) of a booking by PNR.	Functional	High	Passenger	AC-034: Status returned matches the live record, including any waitlist promotions since booking. Test: TC-PNR-01	–

4.6 Admin: Train, Fare & Quota Management

Description: Gives the clerk/admin control over the train master data that the passenger-facing features read from.

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-040	The system shall let an admin add a new train with number, name, route, timings, and per-class seat quota.	Functional	High	Admin	AC-040: A newly added train appears in passenger search results immediately. Test: TC-Admin-01	–
RRS-F-041	The system shall let an admin update an existing train's fare per class and total quota.	Functional	Medium	Admin	AC-041: Updated fare reflects in the next booking's fare calculation. Test: TC-Admin-02	Must not corrupt existing bookings already made at the old fare
RRS-F-042	The system shall let an admin remove or deactivate a train that is no longer in service, blocking new bookings against it while preserving historical records.	Functional	Low	Admin	AC-042: A deactivated train no longer appears in search but its past bookings remain in reports. Test: TC-Admin-03	–
RRS-F-043	The system shall let an admin generate a report of bookings, cancellations and revenue for a given train or date range.	Functional	Medium	Admin	AC-043: Report totals match a manual count over the same file data. Test: TC-Admin-04	Depends on RRS-F-033

4.7 Data Persistence & Recovery

Req ID	Requirement (shall…)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
RRS-F-050	The system shall persist all trains, users and bookings to disk files after every state-changing operation, so no confirmed data is lost if the program exits normally.	Functional	High	Business	AC-050: Restarting the program after a booking shows the same state as before restart. Test: TC-Data-01	–
RRS-F-051	The system shall provide an admin option to back up the current data files to a timestamped folder and restore from a chosen backup.	Functional	Low	Admin	AC-051: Restoring a backup returns the system to that backup's exact recorded state. Test: TC-Data-02	–

5. Non-Functional Requirements

NFRs are written to be measurable against a single-user desktop run, since this is a local C/C++ console application rather than a networked, multi-user service. IDs follow RRS-NF-###.

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
RRS-NF-001	A train search or availability query shall return results within 1 second on a typical development machine, for a data file of up to 5,000 train records.	Performance	High	Manual timing over 20 runs averages ≤ 1s. Test: TC-Perf-01
RRS-NF-002	The application shall not crash or lose committed data when given invalid input (wrong data types, out-of-range values, empty fields).	Reliability	High	A scripted invalid-input pass (≥ 30 malformed inputs) causes zero crashes. Test: TC-Rel-01
RRS-NF-003	Data files shall remain readable and consistent after an abrupt program termination (e.g. window closed mid-write) in at least 95% of induced-failure test runs.	Reliability / Data Integrity	Medium	20 forced-kill tests during a write show ≤ 1 corrupted file. Test: TC-Rel-02
RRS-NF-004	The CLI shall present menus and error messages in plain, unambiguous language understandable to a first-time user without a manual.	Usability	Medium	5 first-time testers complete a booking without external help. Test: TC-UX-01
RRS-NF-005	The codebase shall be organised into separate modules/files by concern (auth, booking, admin, storage) so a new feature can be added without editing unrelated modules.	Maintainability	Medium	Code review confirms module boundaries; adding a sample feature touches ≤ 2 existing files. Test: Code review checklist

5.1 Security

5.1.1 Security Objectives

- Protect stored passenger credentials and personal details from being trivially readable if the data files are opened directly.
- Ensure that only an authenticated admin can alter train, fare and quota data, and that every such change is traceable.

5.1.2 Security Requirements

Req ID	Requirement (shall…)	Type	Priority	Acceptance Criteria / Test Ref
RRS-SR-001	The system shall store passwords in hashed form (not plaintext) in users.dat.	Security	High	AC: opening users.dat in a text/hex viewer shows no readable password. Test: TC-Sec-01
RRS-SR-002	The system shall mask password and dummy card-number input on screen during entry.	Security	High	AC: console echoes '*' instead of typed characters for these fields. Test: TC-Sec-02
RRS-SR-003	The system shall restrict admin-only menu options so they are unreachable without a successful admin login.	Security	High	AC: attempting to reach admin functions pre-login is refused. Test: TC-Sec-03
RRS-SR-004	The system shall log every admin action that changes train, fare or quota data, with a timestamp, to admin_log.txt.	Security	Medium	AC: each such change produces exactly one corresponding log line. Test: TC-Sec-04
RRS-SR-005	The system shall lock a user account for a short cool-down period after 5 consecutive failed login attempts.	Security	Medium	AC: the 6th consecutive failed attempt is refused even with the correct password, until cool-down elapses. Test: TC-Sec-05

6. Quality Attributes & Acceptance Tests

Exit Criteria for Acceptance

All High-priority functional requirements are implemented and pass their reference test case; there are no open critical NFR failures (data loss or crash on valid input); and the RTM in Section 8 shows every listed test case in a Passed state before the build is considered submission-ready.

Acceptance Test Suites

- Registration & Authentication — covers RRS-F-001–003, RRS-SR-002/003/005.
- Search & Availability — covers RRS-F-010–012.
- Booking — covers RRS-F-020–025.
- Cancellation & Refund — covers RRS-F-030–034.
- Admin & Reporting — covers RRS-F-040–043, RRS-SR-004.
- Data Integrity & Recovery — covers RRS-F-050–051, RRS-NF-002/003.

7. System Models and Diagrams

7.1 UML Use-Case Diagram — System Overview

The diagram below shows the two primary actors — the Passenger and the Reservation Clerk/Admin — against the full set of system-level use cases, plus the simulated Payment Gateway as a secondary actor included from the payment step.

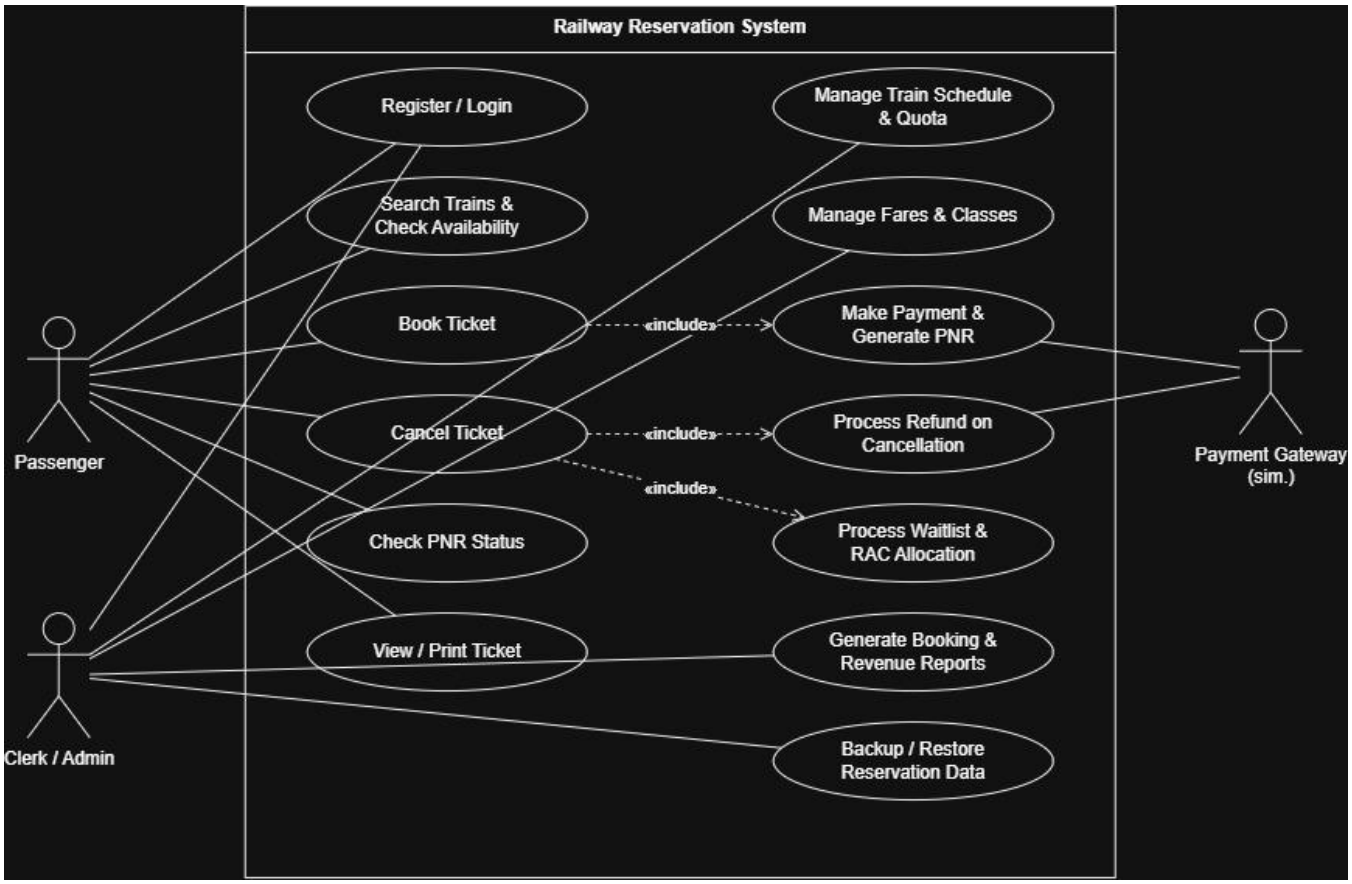


Figure 7.1: Use-case diagram — Railway Reservation System (system-level view)

7.2 UML Use-Case Diagram — Booking & Cancellation Subsystem

This second diagram zooms into the passenger-facing booking and cancellation flow, showing how a full booking decomposes into smaller use cases and how a freed seat extends into automatic waitlist promotion.

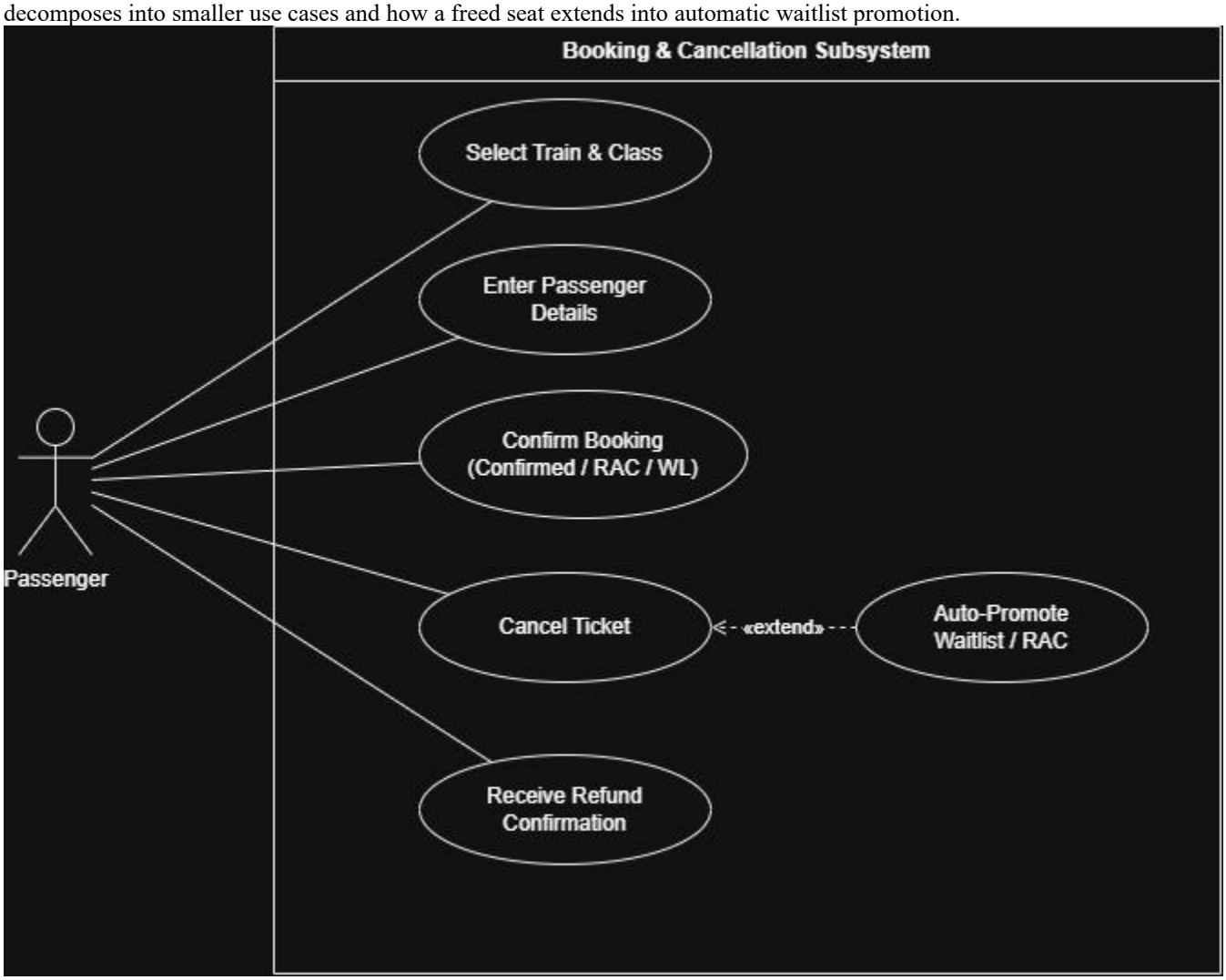


Figure 7.2: Use-case diagram — Booking & Cancellation subsystem detail

8. Requirements Traceability Matrix (RTM)

The RTM below maps each requirement to where it is described in this document, the module expected to implement it, and its reference test case(s). Status uses N (Not started), P (In progress), A (Passed/Accepted) — all are currently N since this SRS precedes implementation.

Req ID	Requirement (short)	Section Ref / Design Spec	Module	Test Case(s)	Status (N/P/A)	Comments
RRS-F-001	Passenger registration	4.1 / DS-Auth-01	AuthModule	TC-Reg-01	N	
RRS-F-002	Passenger login	4.1 / DS-Auth-01	AuthModule	TC-Auth-01	N	
RRS-F-003	Admin login	4.1 / DS-Auth-02	AuthModule	TC-Auth-02	N	
RRS-F-010	Train search	4.2 / DS-Search-01	SearchModule	TC-Search-01	N	

RRS-F-011	Seat availability display	4.2 / DS-Search-01	SearchModule	TC-Search-02	N	
RRS-F-012	WL/RAC hint on full class	4.2 / DS-Search-02	SearchModule	TC-Search-03	N	
RRS-F-020	Passenger detail capture	4.3 / DS-Book-01	BookingModule	TC-Book-01	N	
RRS-F-021	PNR generation	4.3 / DS-Book-02	BookingModule	TC-Book-02	N	Uniqueness-critical
RRS-F-022	Seat decrement on booking	4.3 / DS-Book-02	BookingModule / StorageModule	TC-Book-03	N	
RRS-F-023	RAC/Waitlist queuing	4.3 / DS-Book-03	BookingModule	TC-Book-04	N	
RRS-F-024	Simulated payment	4.3 / DS-Pay-01	PaymentModule (sim.)	TC-Book-05	N	
RRS-F-025	Ticket receipt	4.3 / DS-Book-04	BookingModule	TC-Book-06	N	
RRS-F-030	Cancellation by PNR	4.4 / DS-Cancel-01	CancelModule	TC-Cancel-01	N	
RRS-F-031	Refund slab calculation	4.4 / DS-Cancel-02	CancelModule	TC-Cancel-02	N	
RRS-F-032	Auto waitlist promotion	4.4 / DS-Cancel-03	CancelModule / BookingModule	TC-Cancel-03	N	
RRS-F-033	Cancelled status retained	4.4 / DS-Cancel-04	StorageModule	TC-Cancel-04	N	
RRS-F-034	PNR status lookup	4.5 / DS-PNR-01	BookingModule	TC-PNR-01	N	
RRS-F-040	Add train	4.6 / DS-Admin-01	AdminModule	TC-Admin-01	N	
RRS-F-041	Update fare/quota	4.6 / DS-Admin-02	AdminModule	TC-Admin-02	N	
RRS-F-042	Deactivate train	4.6 / DS-Admin-03	AdminModule	TC-Admin-03	N	
RRS-F-043	Booking/revenue report	4.6 / DS-Admin-04	AdminModule / StorageModule	TC-Admin-04	N	
RRS-F-050	Persist state to disk	4.7 / DS-Store-01	StorageModule	TC-Data-01	N	
RRS-F-051	Backup / restore	4.7 / DS-Store-02	StorageModule	TC-Data-02	N	
RRS-NF-001	Search response time	5 / DS-Perf-01	SearchModule	TC-Perf-01	N	
RRS-NF-002	No crash on bad input	5 / DS-Rel-01	All modules	TC-Rel-01	N	
RRS-NF-003	File consistency after crash	5 / DS-Rel-02	StorageModule	TC-Rel-02	N	
RRS-SR-001	Hashed password storage	5.1.2 / DS-Sec-01	AuthModule	TC-Sec-01	N	
RRS-SR-002	Masked input	5.1.2 / DS-Sec-02	AuthModule / BookingModule	TC-Sec-02	N	

RRS-SR-003	Admin menu access control	5.1.2 / DS-Sec-03	AdminModule	TC-Sec-03	N	
RRS-SR-004	Admin action logging	5.1.2 / DS-Sec-04	AdminModule	TC-Sec-04	N	
RRS-SR-005	Login lockout after failures	5.1.2 / DS-Sec-05	AuthModule	TC-Sec-05	N	